

[Open in app](#)**Medium** Search

Launching my first Aws Ec2 instance with Terraform

7 min read · Jul 16, 2023



Kedar Salunkhe

[Follow](#)

Listen



Share



More



Recently I have started working as a Sr DevOps engineer and as a part of my day 2-day work where I got a chance to work with different cloud providers and automate the boring manual task of creating and destroying resources such as VMs, Users, Storage, cloud provider configurations, etc. So, to automate the daily manual repetitive work I started my journey with one of the IAC tool named terraform.

Through this series of terraform articles, I am going to share my learning with you all. In this article we are going to create an AWS EC2 instance using terraform.

I am using VSCODE editor to write my terraform scripts and the only prerequisite for creating an instance in the aws is that you need to have a user account, so let's start with our steps to create our first EC2 instance.

Before moving to steps the i need to add one thing is that i have not included the installation part of tools needed to setup terraform, aws-cli, vscode. You all can refer youtube or any documentation link for that, but if you want me to add that part too. please do comment and let me know so i can make a seperate article on it.

Steps:-

1. In the vscode editor or whichever editor you prefer, create a folder named [ec2 instance], you can add any name you want. Inside that folder create your terraform file example: - first_ec2.tf.
2. Once you create the .tf file inside the file you need to add the following code.

```
provider "aws" {  
  region      = "ap-south-1"  
  access_key  = "your-access-key"  
  secret_key  = "your-secret-key"  
}
```

To explain the above parameters let's start with the first one.

provider "aws" {} -> the provider aws is used to interact with the resources associated with aws such as iam role, users, s3.

The region,access_key,secret_key are the authentication credentials you can define in terraform.

3. Now we will add the EC2 instance details which will create a new instance for us.

```
resource "aws_instance" "myec2" {  
  ami = "ami-0d13e3e640877b0b9"
```

```
instance_type = "t2.micro"

tags = {
  Name = "ec2-created-from-terraform"
}
```

In the above code you can see the resource is `aws_instance`.

“myec2” -> you can give any name but make sure that you do not change the `aws_instance` as this parameter will help us to create an ec2 instance.

ami -> this is the id of amazon machine image which you select during manual instance creation. You can check the below snapshot for reference.

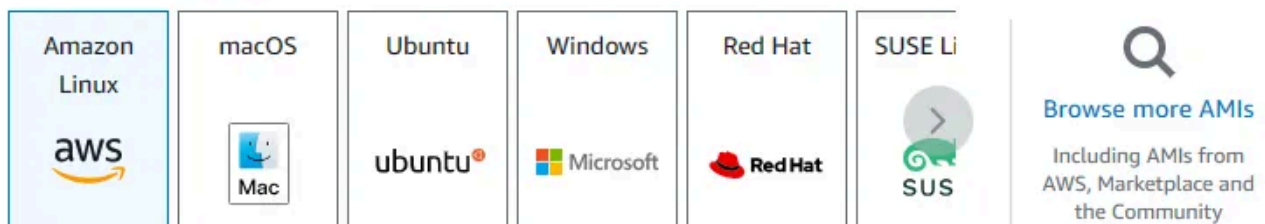
▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

🔍 Search our full catalog including 1000s of application and OS images

My AMIs

Quick Start



Amazon Machine Image (AMI)

Amazon Linux 2023 AMI

ami-0d13e3e640877b0b9 (64-bit (x86)) / ami-0f203c26f765cfb32 (64-bit (Arm))

Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible ▼

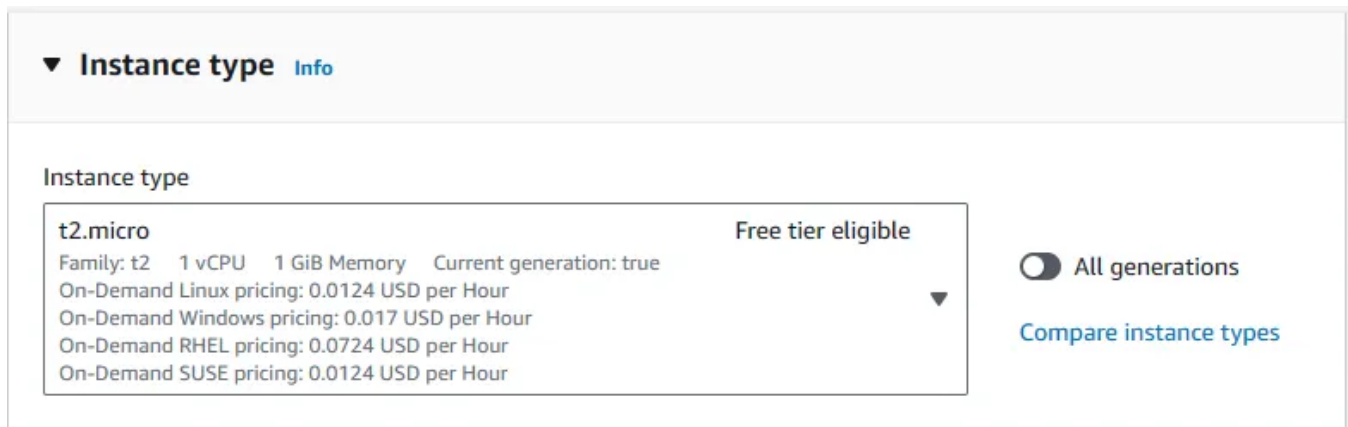
Description

Amazon Linux 2023 AMI 2023.1.20230705.0 x86_64 HVM kernel-6.1

AMI- ami-0d13e3e640877b0b9

`instance_type = "t2.micro"` -> instance type that meets your computing, memory, networking, or storage needs. While manual creation of the instance below is the

GUI window where we select the instance type.



instance type

```

first_ec2.tf > ...
1
2  provider "aws" {
3      region      = "ap-south-1"
4      access_key  = "AKIARIQLXNNQHCLDDNMQ"
5      secret_key  = "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"
6  }
7
8  resource "aws_instance" "myec2" {
9      ami = "ami-0d13e3e640877b0b9"
10     instance_type = "t2.micro"
11
12     tags = {
13         Name = "ec2-created-from-terraform"
14     }
15 }
16

```

4. So this is the full code in the tf file which I used to create my first ec2 instance automatically with use of terraform. Also to mention about the “tags” parameter this is the parameter using which you can give a tag to your instance, can also be called as a short name although there is no compulsion to use this parameter.

5. Now we will proceed towards the execution of the code from the terminal window.

6. Our first step is open the terminal window from the folder so that we are directed to the absolute path in which are tf scripts been located.

A small note:- My base system is Windows from which i am writing the code and executing the terraform scripts.

Run `terraform init` ←- this command should always run from the working directory as it initialized the current working directory consisting of the terraform configuration files and download the plugins associated with the provider. You can see below what happens when i run the `terraform init` command.

```
C:\Users\Admin\Desktop\hasicorp-terraform> terraform init
```

```
Initializing the backend...
```

```
Initializing provider plugins...
```

- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v5.8.0...
- Installed hashicorp/aws v5.8.0 (signed by HashiCorp)

```
Terraform has created a lock file .terraform.lock.hcl to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.
```

```
Terraform has been successfully initialized!
```

```
You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.
```

```
If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.
```

7. Once initialized now run the command `terraform plan`.

this command will give you a preview of what the code written in the terraform file will do, in our scenario our code will create the EC2 instance. You can check from the below output of `terraform plan`.

```
PS C:\Users\Admin\Desktop\hasicorp-terraform> terraform plan
```

```
Terraform used the selected providers to generate the following execution plan.
```

following symbols:

+ create

Terraform will perform the following actions:

```
# aws_instance.myc2 will be created
+ resource "aws_instance" "myec2" {
  + ami                                = "ami-0d13e3e640877b0b9"
  + arn                                = (known after apply)
  + associate_public_ip_address       = (known after apply)
  + availability_zone                  = (known after apply)
  + cpu_core_count                     = (known after apply)
  + cpu_threads_per_core               = (known after apply)
  + disable_api_stop                   = (known after apply)
  + disable_api_termination            = (known after apply)
  + ebs_optimized                      = (known after apply)
  + get_password_data                  = false
  + host_id                            = (known after apply)
  + host_resource_group_arn            = (known after apply)
  + iam_instance_profile                = (known after apply)
  + id                                 = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle                 = (known after apply)
  + instance_state                     = (known after apply)
  + instance_type                      = "t2.micro"
  + ipv6_address_count                 = (known after apply)
  + ipv6_addresses                     = (known after apply)
  + key_name                           = (known after apply)
  + monitoring                         = (known after apply)
  + outpost_arn                        = (known after apply)
  + password_data                      = (known after apply)
  + placement_group                    = (known after apply)
  + placement_partition_number         = (known after apply)
  + primary_network_interface_id       = (known after apply)
  + private_dns                        = (known after apply)
  + private_ip                         = (known after apply)
  + public_dns                         = (known after apply)
  + public_ip                          = (known after apply)
  + secondary_private_ips              = (known after apply)
  + security_groups                    = (known after apply)
  + source_dest_check                  = true
  + spot_instance_request_id           = (known after apply)
  + subnet_id                          = (known after apply)
  + tags_all                           = (known after apply)
  + tenancy                            = (known after apply)
  + user_data                          = (known after apply)
  + user_data_base64                   = (known after apply)
  + user_data_replace_on_change        = false
  + vpc_security_group_ids             = (known after apply)
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the `-out` option to save this plan, so Terraform can't guarantee you run `"terraform apply"` now.

8. Now we will run our final command that is `terraform apply` -> This will apply our desired changes and create an ec2 instance.

```
C:\Users\Admin\Desktop\hasicorp-terraform> terraform apply
```

Terraform used the selected providers to generate the following execution plan. following symbols:

+ create

Terraform will perform the following actions:

```
# aws_instance.myec2 will be created
+ resource "aws_instance" "myec2" {
  + ami                        = "ami-0d13e3e640877b0b9"
  + arn                       = (known after apply)
  + associate_public_ip_address = (known after apply)
  + availability_zone          = (known after apply)
  + cpu_core_count             = (known after apply)
  + cpu_threads_per_core       = (known after apply)
  + disable_api_stop           = (known after apply)
  + disable_api_termination    = (known after apply)
  + ebs_optimized              = (known after apply)
  + get_password_data          = false
  + host_id                    = (known after apply)
  + host_resource_group_arn     = (known after apply)
  + iam_instance_profile        = (known after apply)
  + id                         = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle          = (known after apply)
  + instance_state              = (known after apply)
  + instance_type               = "t2.micro"
  + ipv6_address_count          = (known after apply)
  + ipv6_addresses              = (known after apply)
  + key_name                    = (known after apply)
  + monitoring                  = (known after apply)
  + outpost_arn                 = (known after apply)
  + password_data               = (known after apply)
  + placement_group             = (known after apply)
  + placement_partition_number = (known after apply)
  + primary_network_interface_id = (known after apply)
  + private_dns                 = (known after apply)
  + private_ip                  = (known after apply)
  + public_dns                  = (known after apply)
```

```

+ public_ip                = (known after apply)
+ secondary_private_ips    = (known after apply)
+ security_groups          = (known after apply)
+ source_dest_check        = true
+ spot_instance_request_id = (known after apply)
+ subnet_id                = (known after apply)
+ tags_all                 = (known after apply)
+ tenancy                  = (known after apply)
+ user_data                = (known after apply)
+ user_data_base64         = (known after apply)
+ user_data_replace_on_change = false
+ vpc_security_group_ids   = (known after apply)
}

```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.myec2: Creating...

aws_instance.myec2: Still creating... [10s elapsed]

aws_instance.myec2: Still creating... [20s elapsed]

aws_instance.myec2: Creation complete after 22s [id=i-0da860a00548cb469]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Instances (1) Info									
Find instance by attribute or tag (case-sensitive)									
Instance state = running Clear filters									
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...
☐	ec2-created-from-terraform	i-0da860a00548cb469	Running	t2.micro	2/2 checks passed	No alarms	ap-south-1a	ec2-52-66-214-166.ap-...	52.66.214.166

EC2-INSTANCE CREATED FROM TERRAFORM SCRIPT

You can verify from your EC2 instance console window if the ec2 is been created or not.

So this is how I created my first instance. See you soon in my next learning on terraform. If you feel to share your thoughts on this article or on terraform, please do comment or ping me on my LinkedIn profile.

(2) [Kedar Salunkhe](#) | [LinkedIn](#)

Terraform

AWS

Vscode

Windows 10



Follow

Written by Kedar Salunkhe

63 followers · 10 following

Sr Devops Engineer | Currently exploring AWS, Terraform, K8s, GitOps, Automation

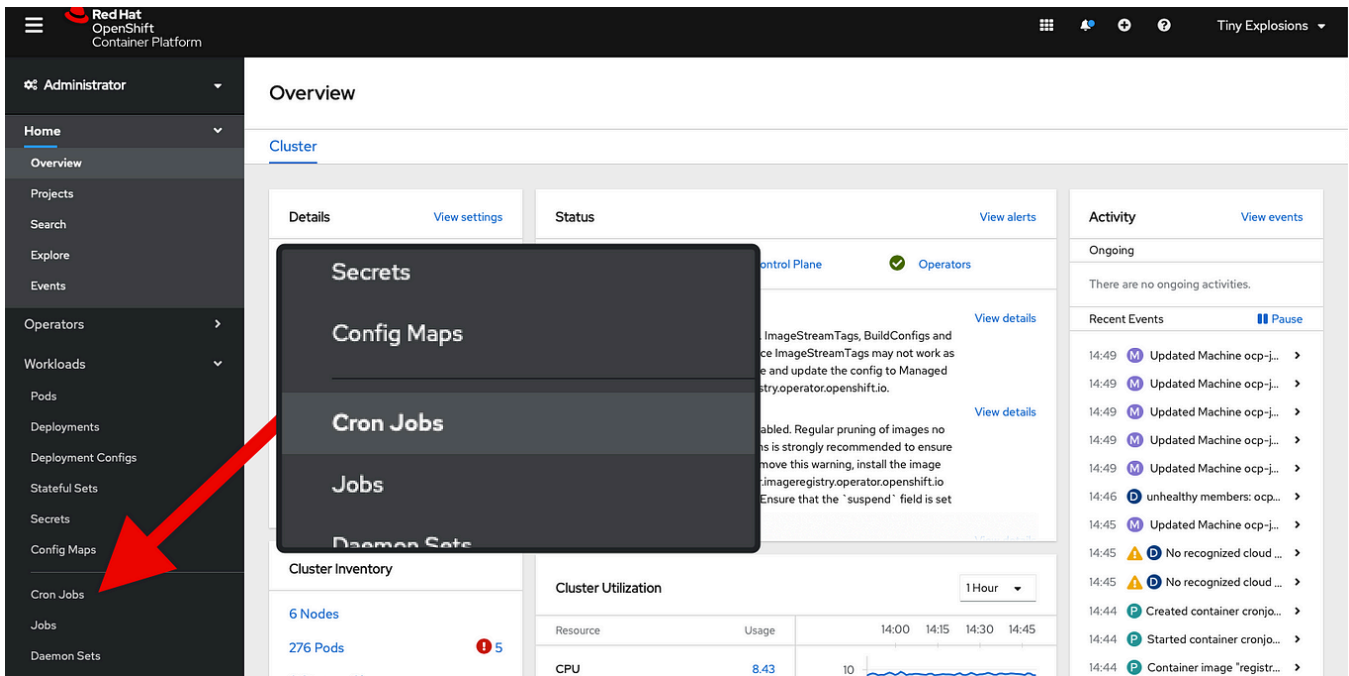
No responses yet



Kwindus

What are your thoughts?

More from Kedar Salunkhe



 Kedar Salunkhe

OpenShift Cronjob for scheduling Deployments Restart

Scheduling a cronjob in OpenShift sometimes could be a little complex task as their may be situations where the node on which we will...

Jan 9, 2023  8



 Kedar Salunkhe

Prometheus installation on EC2 instance

In this article, we will cover how to get Prometheus up and running with systemd on a EC2 server.

Dec 28, 2023 10 1



Pipelines

[Pipelines](#) [PipelineRuns](#) [PipelineResources](#) [Conditions](#) [Repositories](#)

Name	Status	Task status	Started	Duration
PLR build-and-deploy-run-24hwn	✓ Succeeded		Apr 2, 2023, 3:48 PM	2 minutes
PLR build-and-deploy-run-6lbt6	✓ Succeeded		Apr 2, 2023, 3:31 PM	1 minute 5

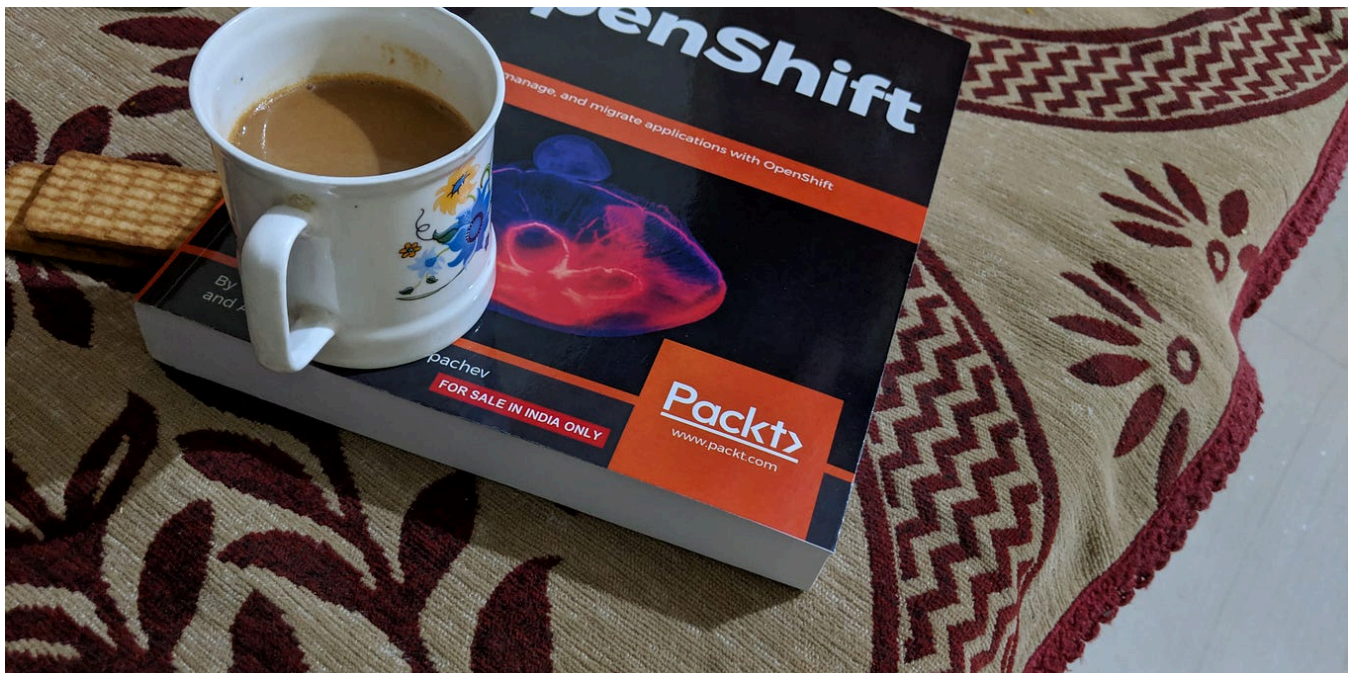


Kedar Salunkhe

Creating my first Red Hat Openshift Pipeline with TEKTON

As working on Openshift from the platform side I always had curiosity of how the app team goes through application deployment, CI/CD process...

Apr 2, 2023 1



Kedar Salunkhe

Troubleshooting the Readiness probe / Liveness Probe error

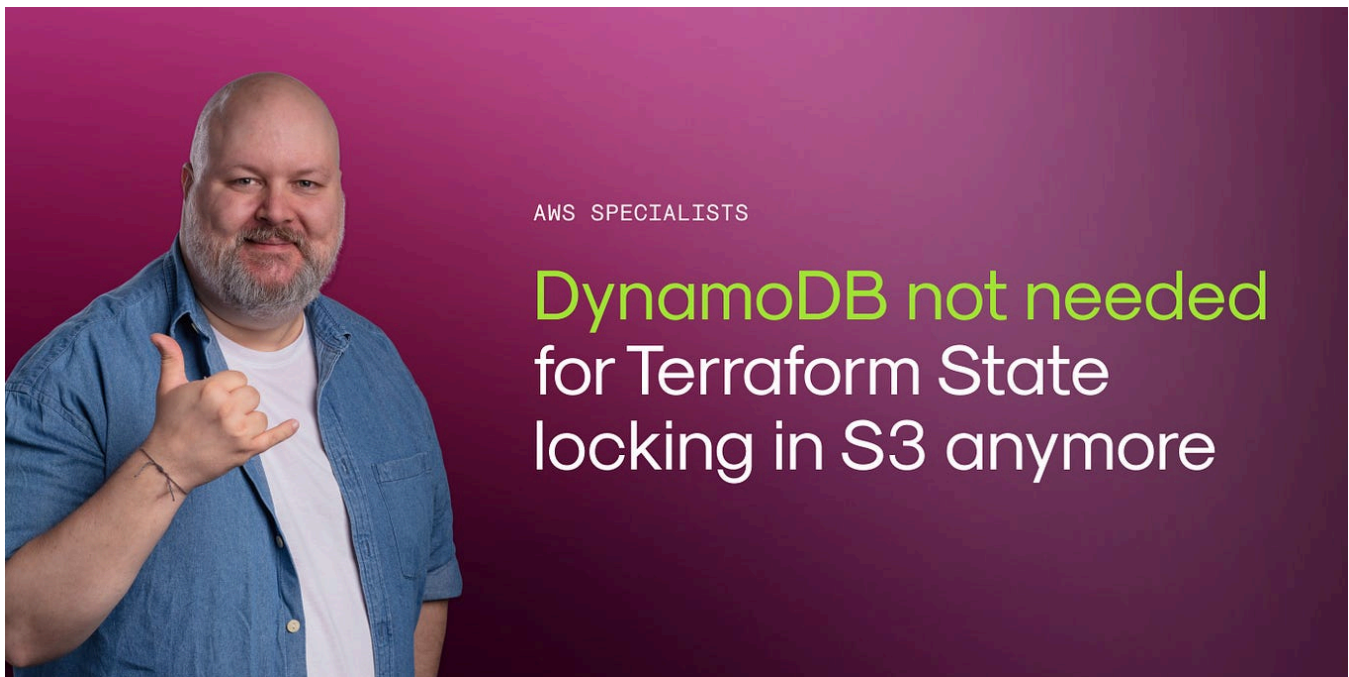
On a typical office working hour I came across this issue of Readiness/Liveness probe error which was constantly making the Openshift pod...

Dec 11, 2022 🖱 4



See all from Kedar Salunkhe

Recommended from Medium



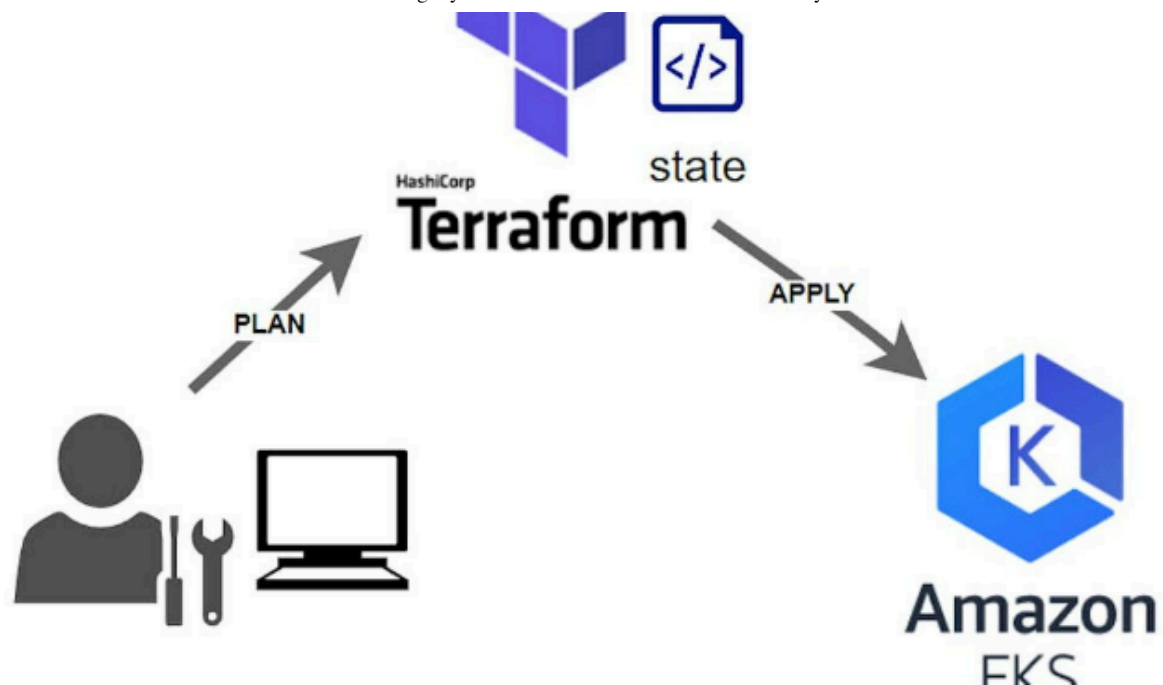
In AWS Specialists by Jens Båvenmark

DynamoDB not needed for Terraform State locking in S3 anymore

Terraform 1.11.0 introduces S3-native state locking, eliminating the need for DynamoDB. Learn how to simplify your setup.

Mar 10 🖱 13





 Vinay K N

Provisioning an EKS Cluster Using Terraform

In this guide, you'll learn how to provision an Amazon EKS (Elastic Kubernetes Service) cluster using Terraform, a powerful...

5d ago



 Jeffrey Taylor

From Notebook to Model Server: Automating MLOps with Ansible, MLflow, and Argo Workflows

A Practical Guide to Full-Stack MLOps on Kubernetes with Infrastructure-as-Code

Apr 24



Sampark Mehrotra

How to Configure Terraform AWS Provider with IAM Role Assumption

In secure AWS environments or multi-account setups, direct access credentials are often avoided. Instead, teams rely on IAM Role assumption...

Apr 9



4

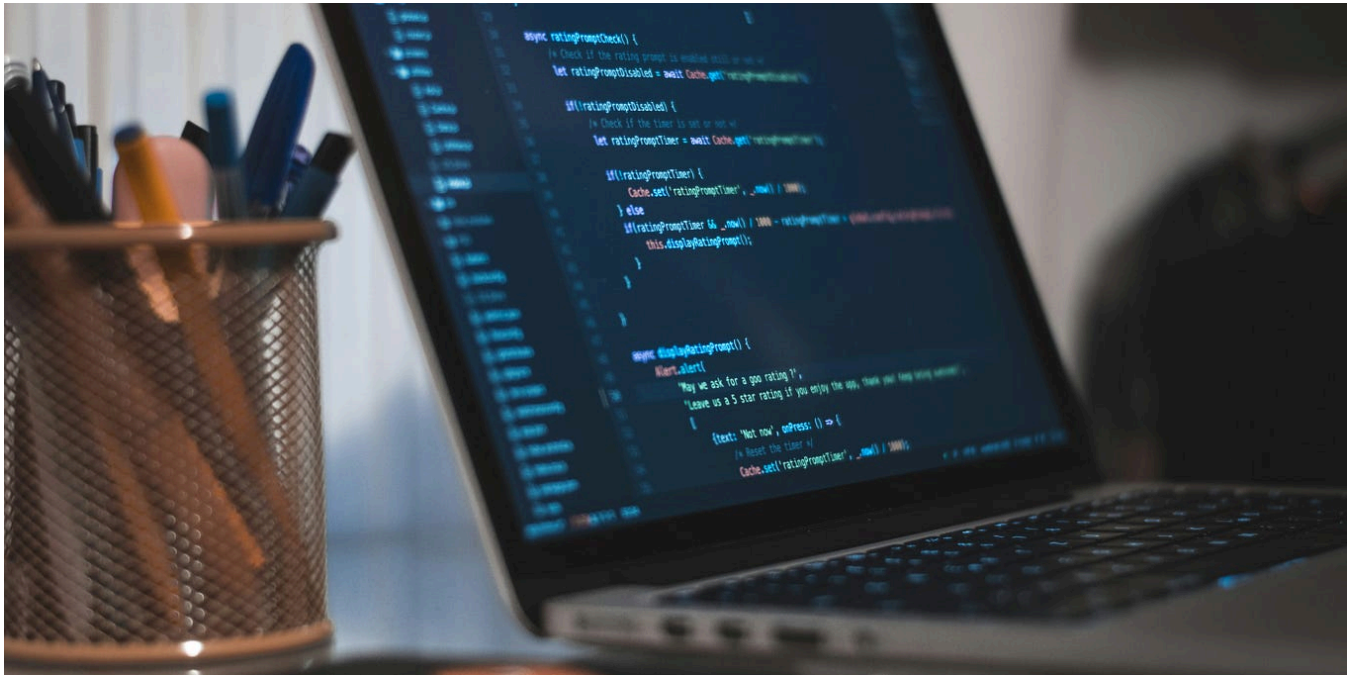


Mihir Popat

Common Issues While Executing Ansible Playbooks

Ansible is a powerful automation tool that simplifies configuration management, application deployment, and infrastructure orchestration...

Feb 26 🖱️ 3



In T3CH by Maciej

Terraform Import Block: Import Existing Resources to Terraform

In the world of Infrastructure as Code (IaC), Terraform has become one of the most widely used tools for automating infrastructure...



Mar 4 🖱️ 51



See more recommendations